MKEC1123 – Advance Microprocessor System

Group Assignment

Group Name:

Lee Jia Xin MKE251045

Hui Jiacong MKE251040

Mohammad Azwan Safwan Bin Harun MKE251024

Prepared For:

Dr Mohd Afzan Othman

Title: Milestone 1 - Familiarizing with the STM32F4 Development Board

Introduction:

The primary focus of this initial milestone was to establish familiarity with the STM32F4 Development Board and its associated Integrated Development Environment (IDE). Before proceeding to more complex firmware development, it was critical to verify that the hardware and software development toolchains were fully operational and properly communicating with the target hardware.

The "Blinky" application was designed to toggle (blink) a physical onboard LED at a designated time interval.

Hardware Components and Specifications:

A NUCLEO-F446RE development board was utilized for the implementation of this project. A mini-USB cable was used to establish communication and provide power to the board. Additional hardware components, including sensors, LEDs, and switches, were employed to facilitate data acquisition, user input, and output indication.

Software Requirements:

To establish the development environment for the board and facilitate future software engineering tasks, a dedicated compiler toolchain was required.

The following software tools were selected and utilized for the project:

- STM32CubeIDE:
  Employed as the primary integrated development environment (IDE) for code compilation, writing, and debugging.
- STM32CubeMX:
  Utilized for the graphical configuration of initialization code, peripherals, and clock settings.

Methodology and Procedure:

1. The STM32CubeMX software was opened to initiate the hardware configuration.
2. The File menu was accessed from the top menu bar, and New Project was selected to launch the target selection window.

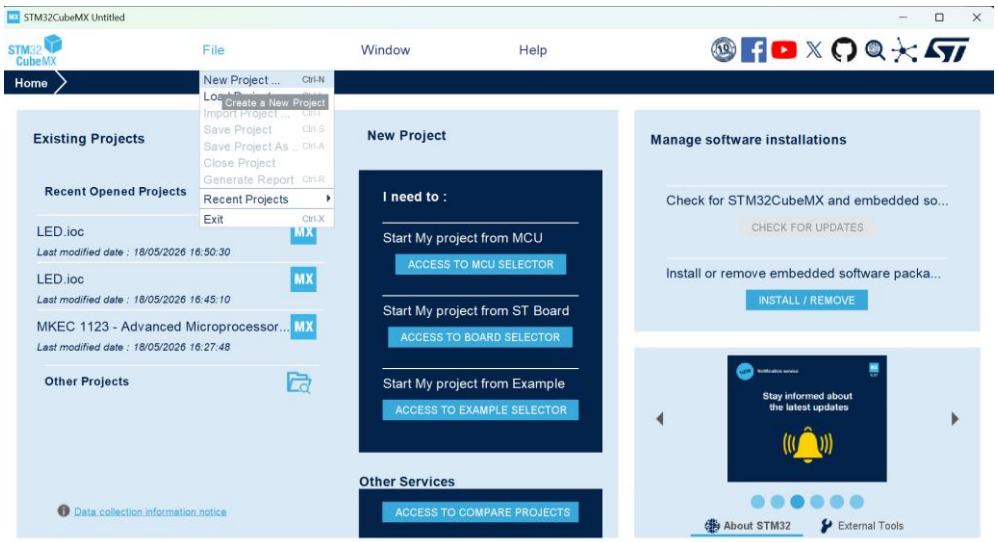

Figure 1: STM32CubeMX home screen showing the initialization of a new project.

3. The Board Selector tab was selected, and the specific commercial part number NUCLEO-F446RE was entered into the search filter field. The target board was then selected from the matching items list at the bottom of the interface to load its specific hardware configuration.

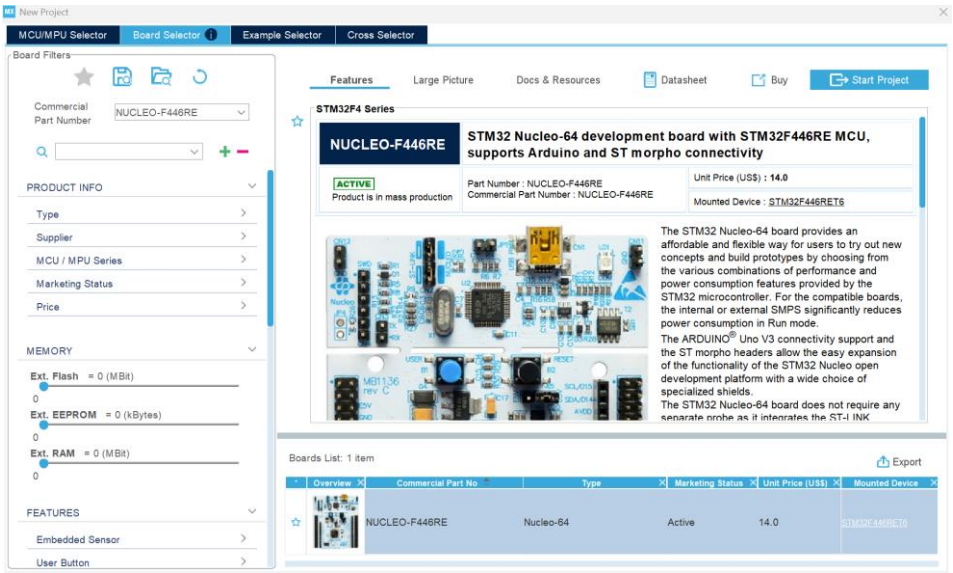

Figure 2: Board selector interface in STM32CubeMX indicating the selection of the NUCLEO-F446RE development board.

4. After verifying the board selection, the Start Project button was clicked in the upper right corner to initialize the peripheral configuration and pinout assignment interface for the microcontroller.

5. Following initialization, the Pinout & Configuration graphical interface was automatically displayed, presenting the default pin layout and pre-configured peripheral assignments specific to the NUCLEO-F446RE development board.

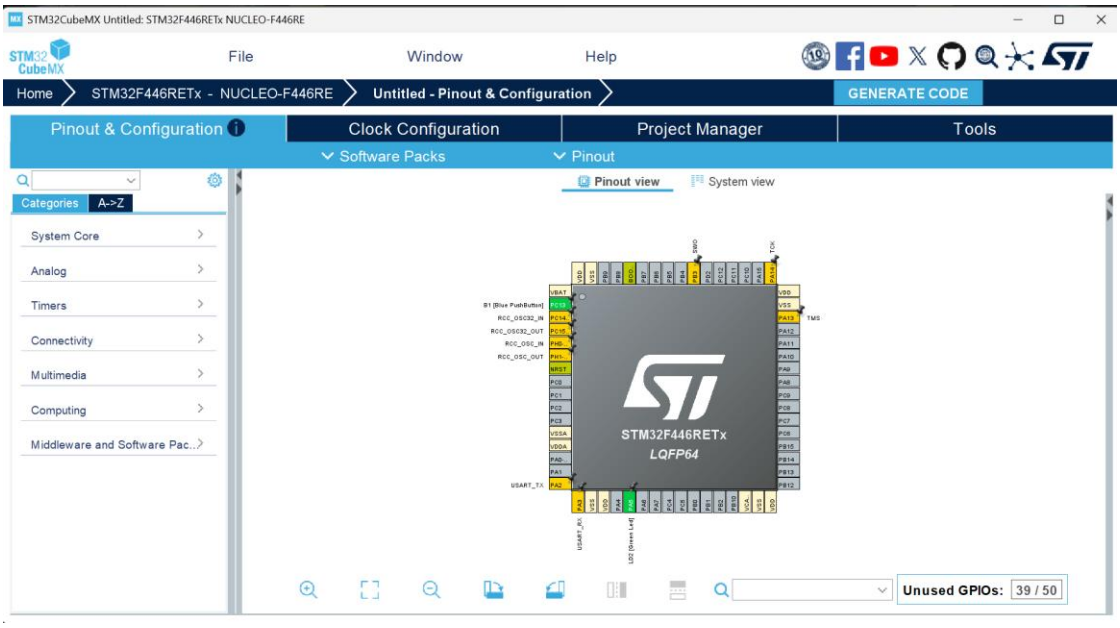

Figure 3: Pinout and Configuration view in STM32CubeMX displaying the default pin mapping for the STM32F446RETx microcontroller.

6. Prior to saving, a dedicated workspace folder named "Blinky LED" was created within the local directory to store all project assets. The File menu was then navigated, and Save Project was selected to save the initial microcontroller hardware configuration file (.ioc) directly inside that newly established folder.

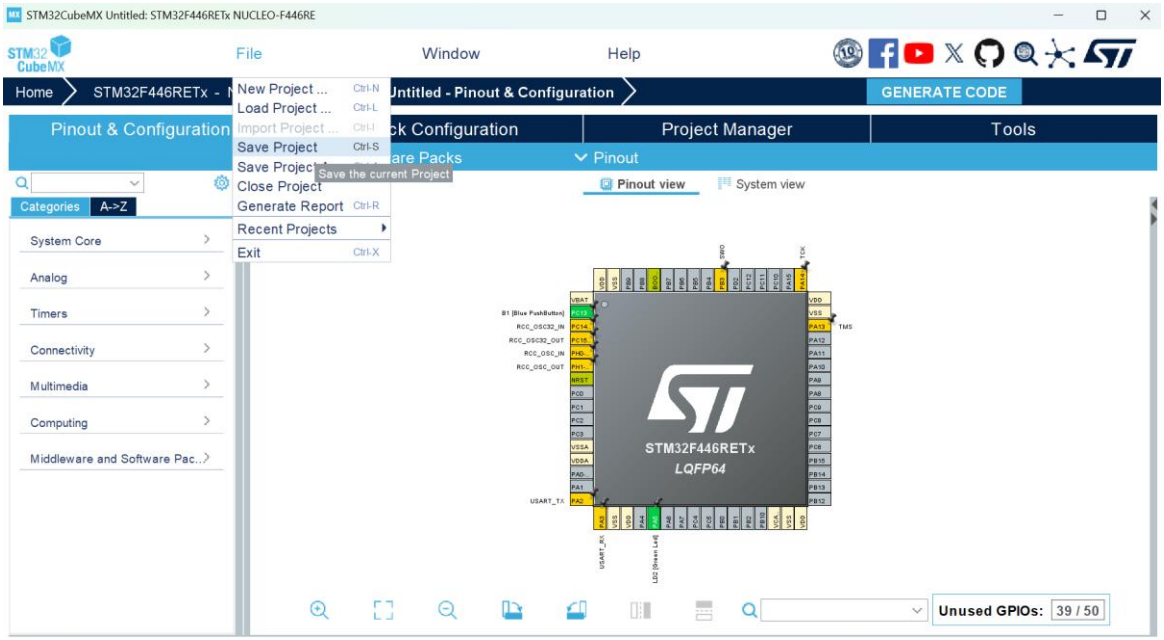

Figure 4: Saving the project configuration file via the File menu in STM32CubeMX.

7. The Project Manager tab was selected to configure the generation settings. The Project Name was designated as "Blinky LED," and the destination Project Location field was verified. Additionally, the Toolchain / IDE dropdown menu was set to STM32CubeIDE to ensure compatibility with the selected development environment.

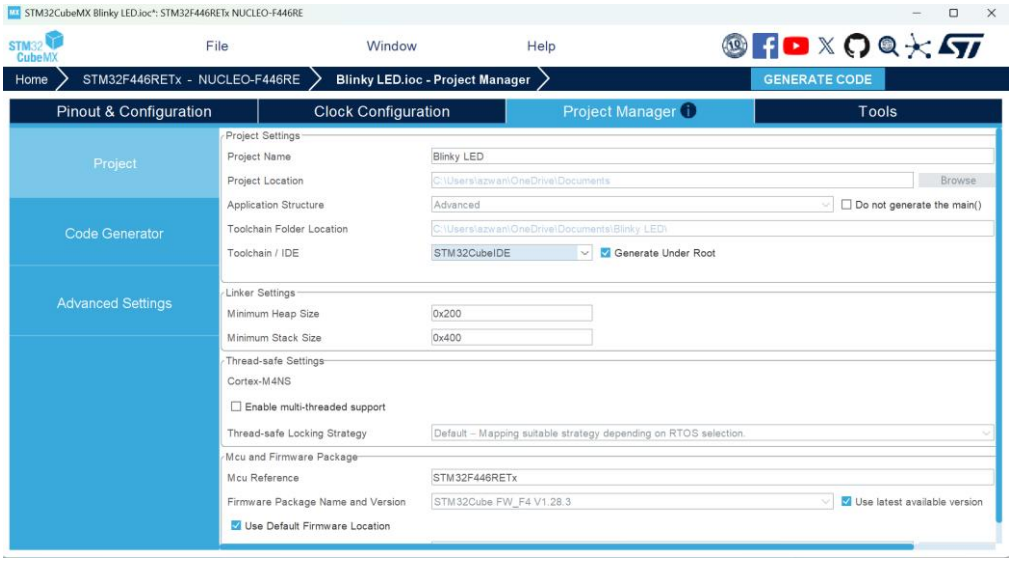

Figure 5: Configuring project settings and toolchain selection within the Project Manager tab in STM32CubeMX.

8. Once the project configuration settings were finalized, the GENERATE CODE button located in the upper-right corner of the interface was clicked to automatically produce the peripheral initialization C source code and structure the project directories for the IDE.
9. A confirmation dialog box appeared, indicating that the source code had been successfully generated under the designated directory path. The Open Project button was then clicked to launch the project directly within STM32CubeIDE.

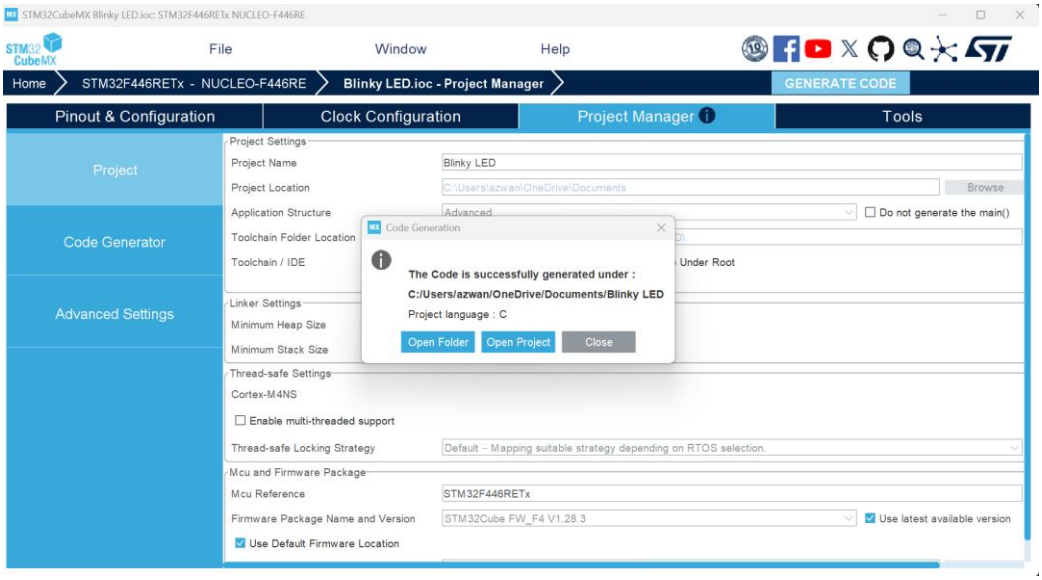

Figure 6: Code generation confirmation dialog box indicating successful project creation.

10. By selecting the Open Project option, the STM32CubeIDE workspace was automatically launched, and the generated initialization files and project directory structure were loaded into the development environment for firmware programming.
11. Upon launching STM32CubeIDE, an "Operation completed" dialog box appeared, confirming that the 'Blinky LED' project was successfully imported and initialized within the IDE workspace. The OK button was clicked to proceed to the source code.

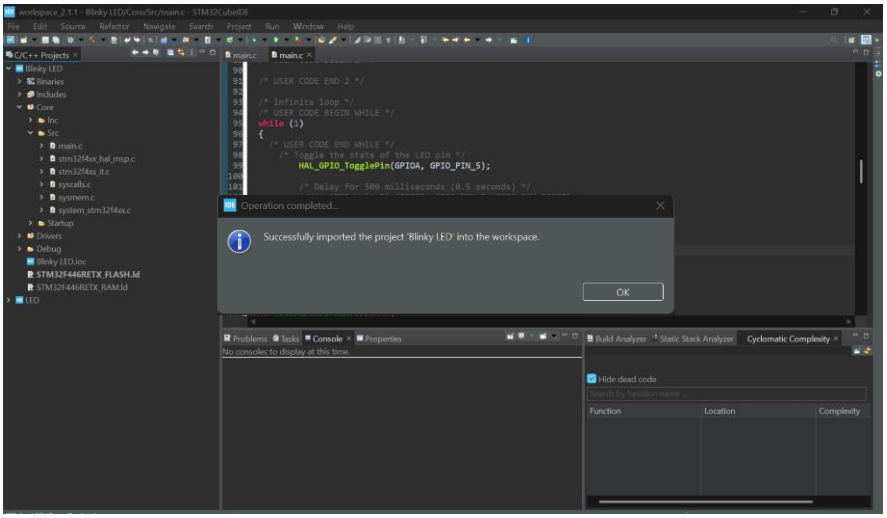

Figure 7: Operation completed dialog in STM32CubeIDE confirming the successful import of the generated project.

12. After clicking OK to dismiss the dialog box, the project directory structure became fully accessible. The main.c source file, located within the Core/Src directory in the Project Explorer panel, was then opened to begin writing the firmware for the Blinky application.
13. The infinite while (1) loop within main.c was navigated to insert the application logic. At line 99, the function HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5); was added to toggle the state of the designated General Purpose Input/Output (GPIO) pin driving the onboard LED. Following this, at line 103, a 2000-millisecond execution delay was implemented by adding the HAL_Delay(2000); function call, thereby establishing a two-second interval between LED state changes.

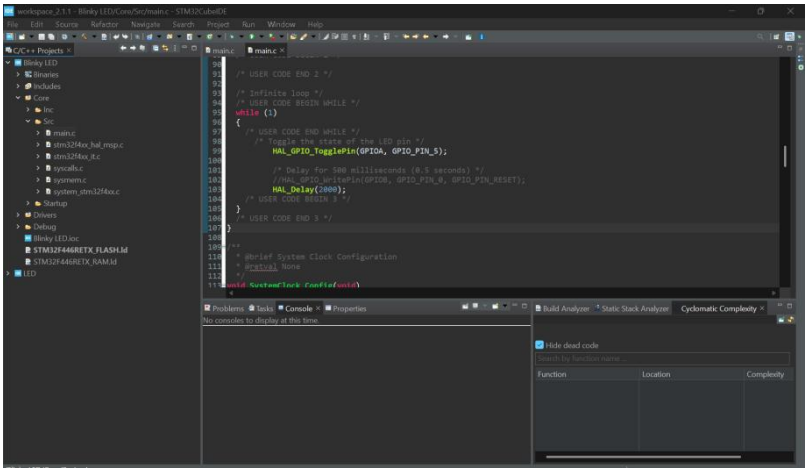

Figure 8: Implementation of the LED toggling logic and time delay functions inside the while(1) infinite loop of main.c.

14. Following the code modifications, the Project menu from the top navigation bar was accessed, and the Clean... command was executed. This operation was performed to explicitly delete all previously compiled object files and old build artifacts from the directory. Doing so ensured that the subsequent build process would force a complete, fresh recompilation of the updated source code, preventing any potential conflicts or errors from cached files.

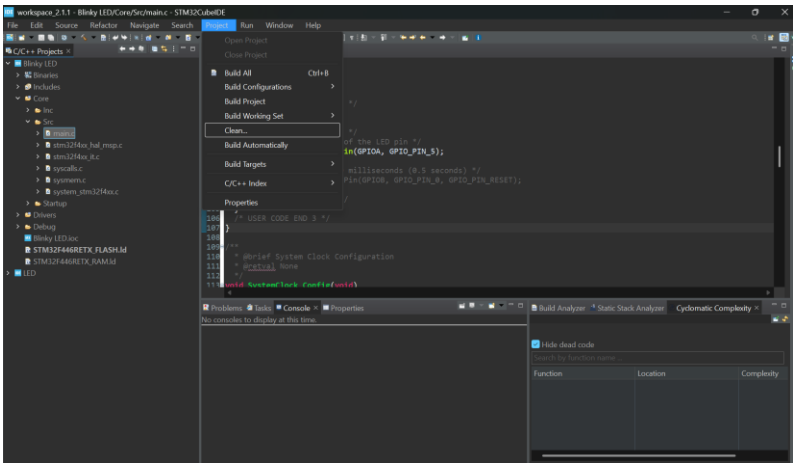

Figure 9: Executing a project clean operation via the Project menu in STM32CubeIDE to clear previous build artifacts.

15. Within the resulting Clean dialog box, the Clean all projects checkbox was selected, and the options to Start a build immediately for the entire workspace were verified. The Clean button was then clicked to execute the command, which discarded all previous build states and automatically initiated a fresh compilation of the updated codebase from scratch.

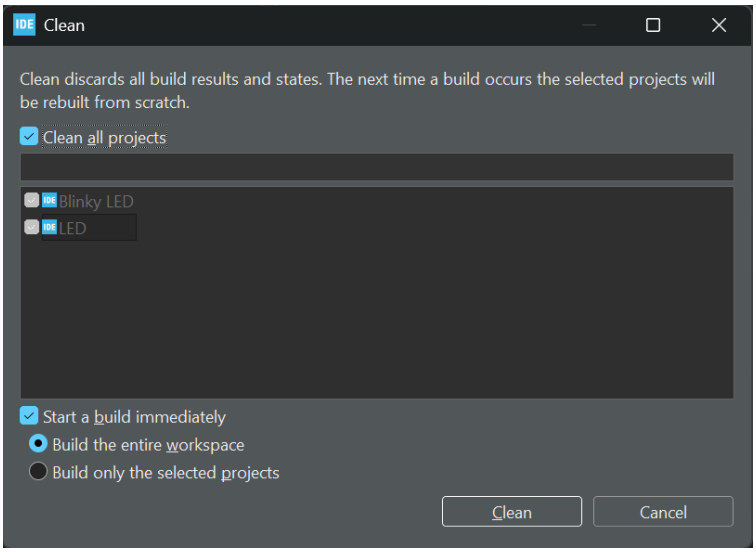

Figure 10:The Clean configuration dialog window in STM32CubeIDE configured to purge and automatically rebuild the entire workspace.

16. Following the rebuild process, the Console tab located at the bottom of the workspace interface was inspected to verify the compilation outcomes. The build log was reviewed to ensure that the source code compiled successfully, explicitly confirming a status of 0 errors and 0 warnings, which validated that the executable binary was ready for hardware deployment.

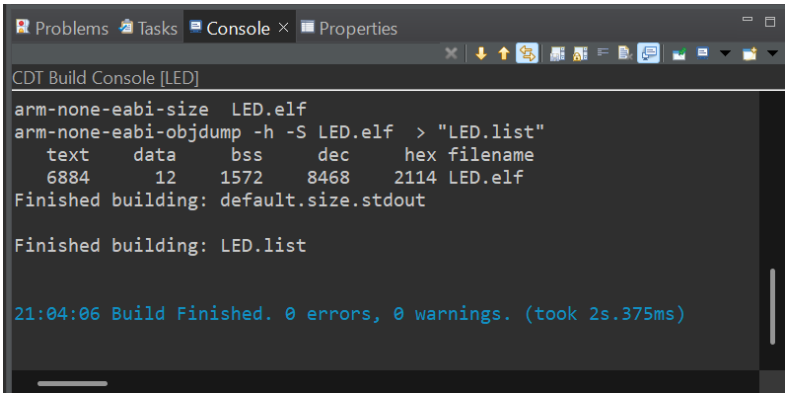

Figure 11: STM32CubeIDE build console output displaying a successful compilation with zero errors and zero warnings.

17. Following the cleaning process, the Project menu was accessed once again, and the Build All command was selected. This action triggered the toolchain's compiler and linker to process the updated main.c file and its associated dependencies, creating the final executable binary file ready for flashing onto the microcontroller target hardware.

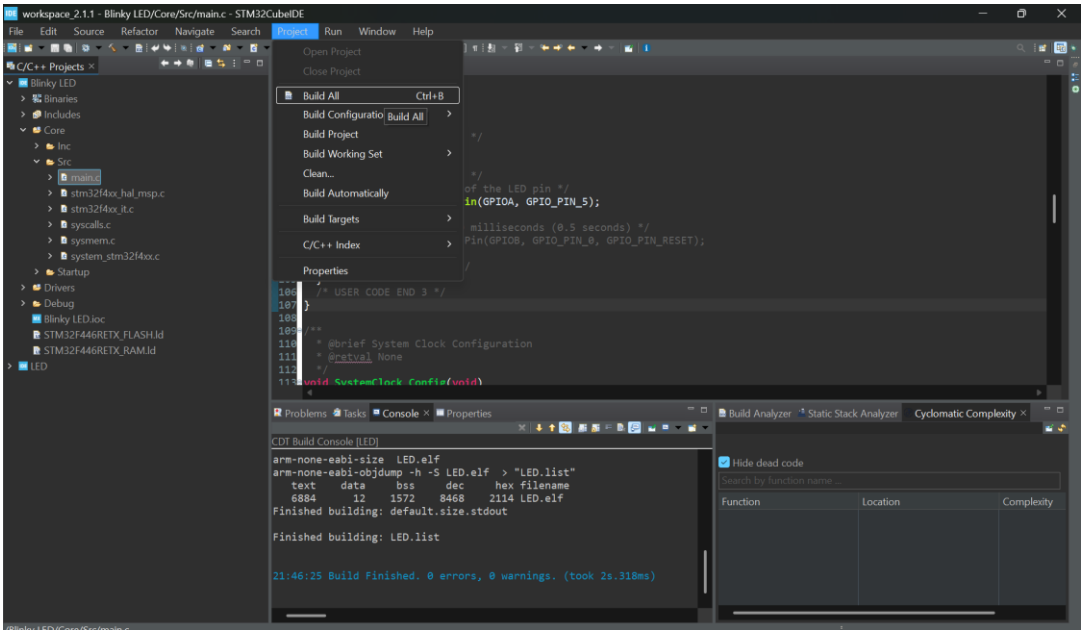

Figure 12: Compiling the finalized firmware source code using the Build All command under the Project menu in STM32CubeIDE.

18. After clicking Build All, the compiler fully reprocessed the code workspace. The CDT Build Console at the bottom verified that the incremental build successfully completed in 189ms with 0 errors and 0 warnings, confirming that the new project structure and main.c modifications are perfectly compiled and ready to be flashed onto the hardware.

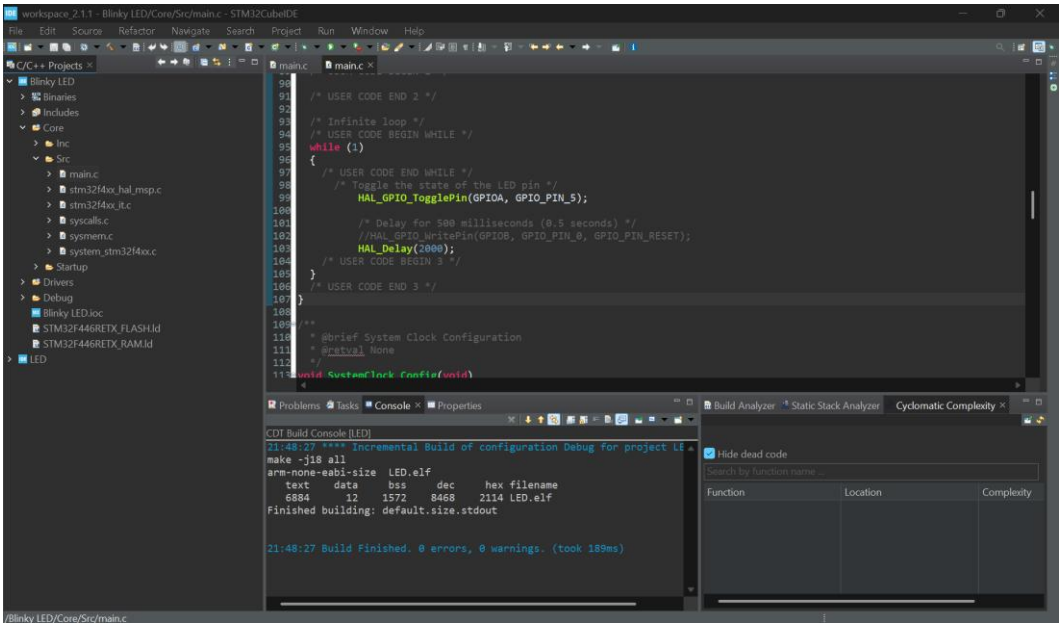

Figure 13: Successful compilation logs in STM32CubeIDE showing build completion after executing the "Build All" command.

19. Connected the STM32 board to the laptop.

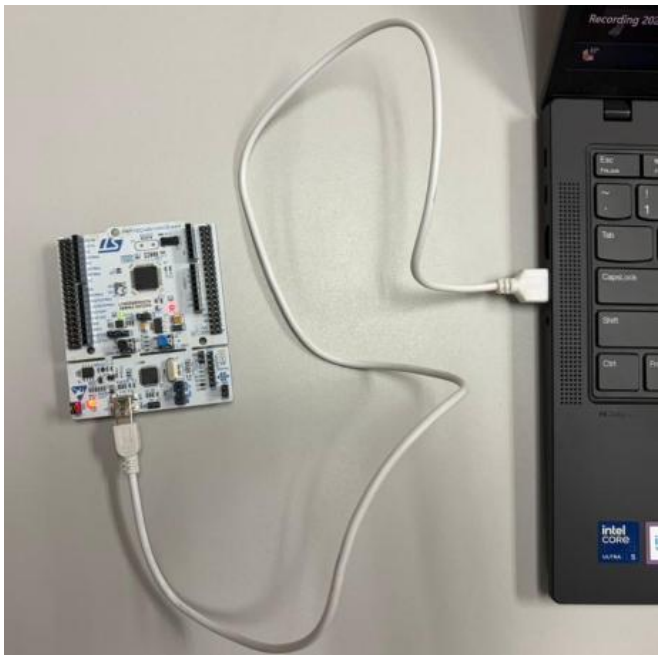

Figure 14: STM32 Nucleo development board connected to a laptop via USB cable.

20. The STLINKV2-1 (D:) window appeared in File Explorer after connecting the board to the laptop.

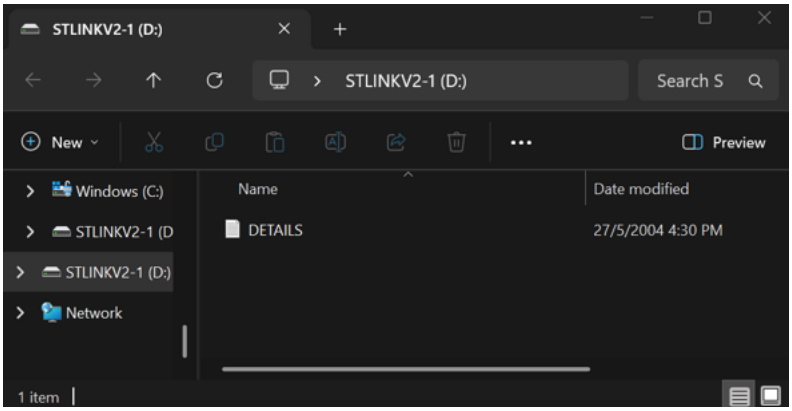

Figure 15: STLINKV2-1 mass storage drive opened in Windows File Explorer.

21. After confirming a successful build with no compilation errors, the Help menu on the top menu bar was selected, and the ST-LINK Upgrade option was clicked. This step was initiated to open the ST-LINK firmware upgrade utility, ensuring that the onboard programmer/debugger firmware on the Nucleo board is fully up to date before uploading the compiled binary to the hardware.

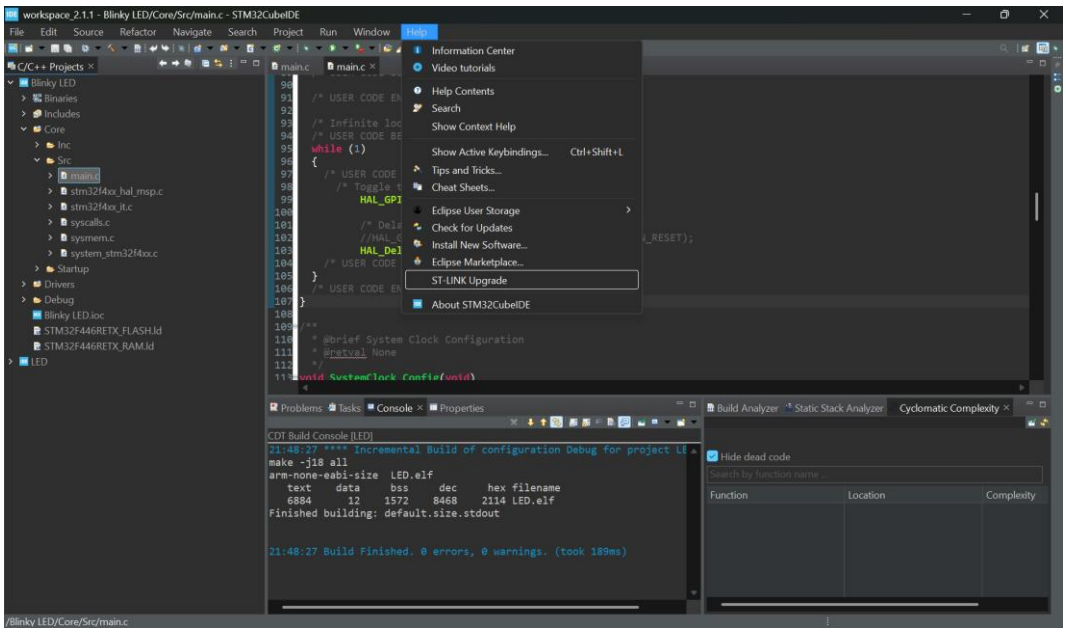

Figure 16: Accessing the ST-LINK Upgrade tool through the Help menu in STM32CubeIDE.

22. Opened the STLinkUpgrade tool, and the firmware upgrade application window appeared.

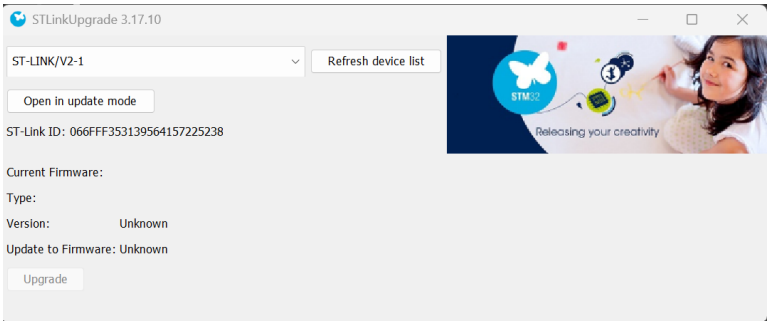

Figure 17: STLinkUpgrade utility window displaying the connected ST-LINK/V2-1 device.

23. Clicked the "Open in update mode" button.
24. Clicked the "Upgrade" button to finalize the firmware update process.
25. Verified that the message "Upgrade successful." appeared at the bottom of the window, confirming the firmware update was complete.

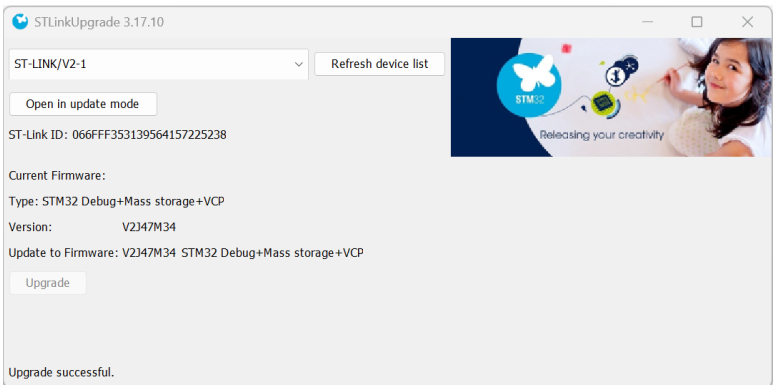

Figure 18: STLinkUpgrade window displaying the "Upgrade successful" status message.

26. Verified the successful execution of the demo by observing the LED blinking at 2-second intervals, as measured by a stopwatch.

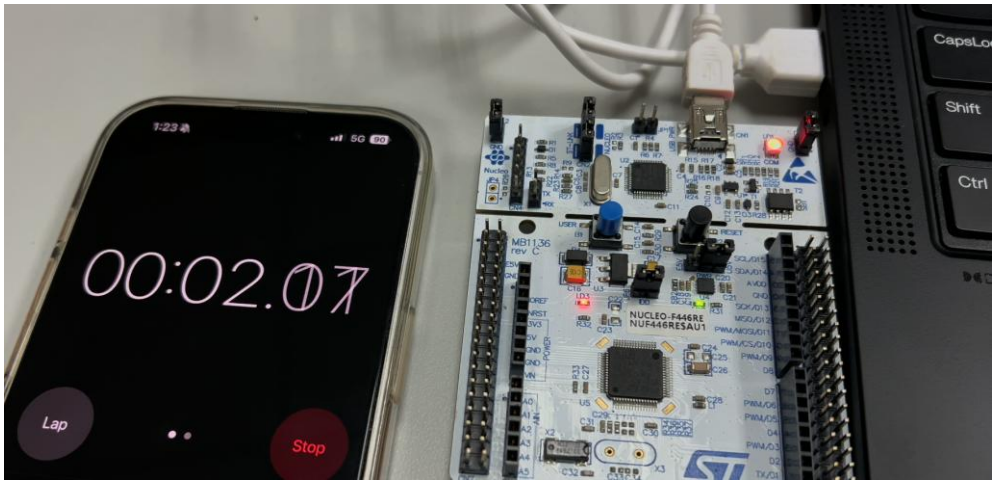

Figure 19: Smartphone stopwatch used to verify the 2-second LED blinking interval on the STM32 Nucleo board.